CSE 11: Introduction to Programming and Computational Problem Solving Accelerated Pace

Assignment 5

Objects and Classes

Due Date: Thursday, November 14, 11:59 PM

Learning goals:

- Implement a program by breaking it up into classes.
- Write unit tests using objects and instance methods.

NOTE: This programming assignment must be done individually.

Your grade will be determined by your most recent submission. If you submit to Gradescope after the deadline, it will be marked late and the late penalty will apply regardless of whether you had past submissions before the deadline.

If your code does not compile on Gradescope and the autograder fails to execute properly, you will receive an automatic zero on the autograder portion of the assignment. Make sure that you do not change the function signatures or import other packages unless otherwise specified.

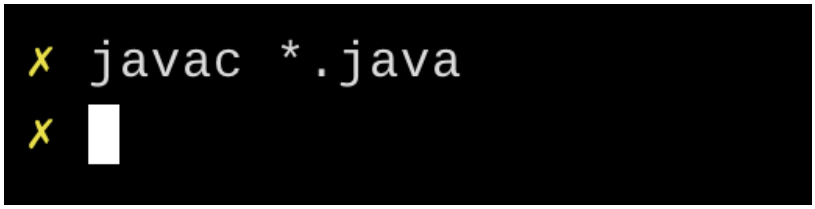
Coding Style (10 points)

For this programming assignment, we will be enforcing the CSE 11 Coding Style Guidelines. These guidelines can also be found on Canvas and the class website. Please ensure to have *COMPLETE* file headers, class headers, and method headers, use descriptive variable names and proper indentation, and avoid using magic numbers.

Part 0: Getting the Starter Code (0 points)

1. If using a personal computer, then ensure your Java software development environment does not have any issues. If there are any issues, then review Assignment 1, or come to the office/lab hours before you start Assignment 5.
2. First, navigate to the `cse11` folder you created in Assignment 1 and create a new folder named `pa5`

3. Download the starter code. You can download the starter code from Piazza → Resources → Homework → `PA5.zip`. The starter code contains three files: `Post.java`, `DynamicArray.java`, and `DiscussionBoard.java`. Place the starter code within the `pa5` folder you just created.
4. From within the `pa5` folder, you can compile all files using the single command `javac *.java`. Doing so should result in **zero compiler errors:**



Overview

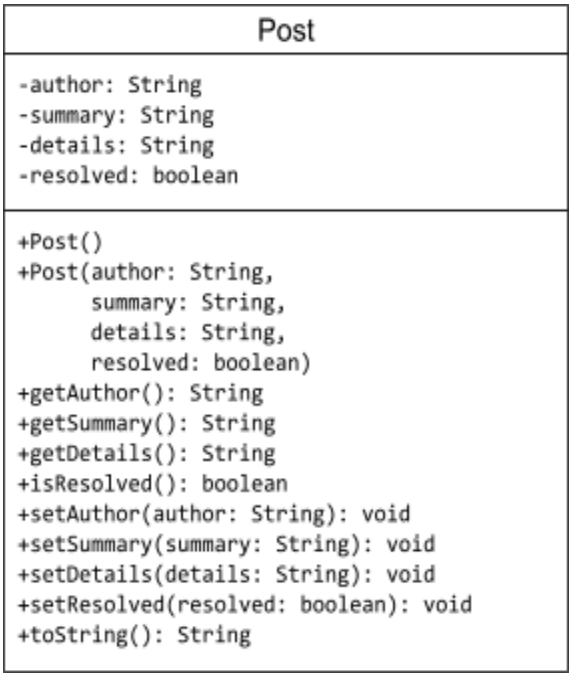
In this programming assignment, you are tasked with developing a discussion board program that works a lot like the one you've been using in this course! (Piazza). This discussion board program has the following three components:

- Part 1: Class `Post` - represents properties of a single post
- Part 2: Class `DynamicArray` - represents an array that can expand in capacity
- Part 3: Class `DiscussionBoard` - represents a discussion board where `Post`'s are organized into a `DynamicArray`

Part 1: Post.java (20 points)

In this part of the assignment, you will implement a `Post` class that represents the properties of a single discussion board post.

The UML diagram for the `Post` class is:



To summarize, the `Post` class contains the following fields:

`private String author`

- The name of the person who made this `Post`

`private String summary`

- A brief overview of the topic of this `Post`

`private String details`

- The description of this `Post`

`private boolean resolved`

- `true` if this `Post` is marked as "resolved" and `false` otherwise.

Notice how each field is declared as `private`. This means that each field is only visible from *within* the (`Post`) class, not from any other class. In other words, you will need to use accessors (i.e., getter methods) and mutators (i.e., setter methods) to access and modify, respectively, these `private` fields when working with `Post` objects from within other classes.

The `Post` class also contains the following instance methods:

`public Post()`

- This constructor must: set `author` to `null`, set `summary` to `null`, set `details` to `null`, and set `resolved` to `false`.

```
public Post(String author, String summary, String details, boolean
        resolved)
```

- This constructor must set the `author`, `summary`, `details`, and `resolved` fields of the `Post` object being created with the values from the parameters (i.e., the `author` *field* must be assigned the contents of the `author` *parameter*)

`public String getAuthor()`

- Return the `author` of this `Post`

`public String getSummary()`

- Return the `summary` of this `Post`

`public String getDetails()`

- Return the `details` of this `Post`

`public boolean isResolved()`

- Return `true` if this `Post` is marked as “resolved” and `false` otherwise.

`public void setAuthor(String author)`

- Update the `author` field of this `Post` with the value of the `author` parameter

`public void setSummary(String summary)`

- Update the `summary` field of this `Post` with the value of the `summary` parameter

`public void setDetails(String details)`

- Update the `details` field of this `Post` with the value of the `details` parameter

`public void setResolved(boolean resolved)`

- Update the `resolved` field of this `Post` with the value of the `resolved` parameter

`public String toString()`

- Return a String representation of this `Post`
- The string representation of a `Post` object must follow the format:

  ```
  Author:\t[author]\n
  Summary:\t[summary]\n
  Details:\t[details]\n
  Resolved?:\t[resolved]\n
  ```

- Where `[field]` should be populated with the corresponding field value of this `Post`. **We strongly recommend that you use the provided format string template (along with the `String.format()` method) in your implementation of this method.**

- **Example:** assume we have a `Post` object called `post`, whose `author` field is `Alice`, `summary` field is `PA5: Stuck on Post toString()`, `details` field is `Struggling to understand how String.format() works`, and `resolved` field is `false`. Then, calling `post.toString()` should return the following string:

  ```
  Author:\tAlice\n
  Summary:\tPA5: Stuck on Post toString()\n
  Details:\tStruggling to understand how String.format() works\n
  Resolved?:\tfalse\n
  ```

- **Implementation Notes**:
  - `\t` is a tab character.
  - `\n` is a newline character.
  - You should not include brackets (i.e., `[]`) in the string you return.

Part 2: DynamicArray.java (30 points)

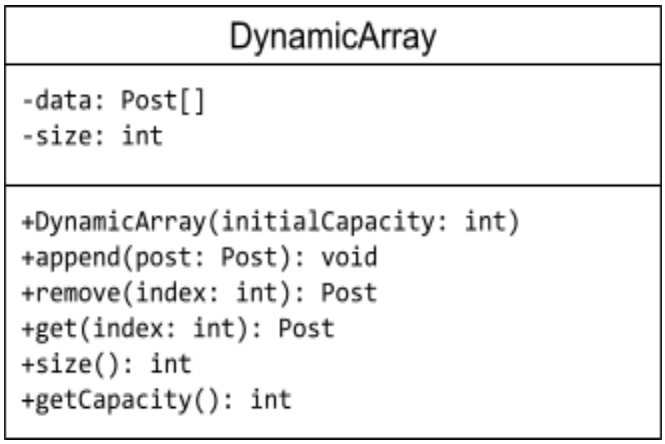
In this part of the assignment, you will implement a `DynamicArray` class which represents an array that can *expand* in length once its maximum capacity is reached. Recall that for built-in Java arrays, once they are created, their capacity is **fixed.**

```
int[] array = new int[10]     // Fixed capacity of 10
```

There is no way to add more than 10 elements to the array referenced by `array`; The only work-around would be to initialize a `new` array object with a larger capacity. A `DynamicArray` object allows you to store items just like a built-in java array *but* with the added capability of being able to *expand* in capacity without needing to initialize a new `DynamicArray` object.

Notably, the `DynamicArray` class you implement will only be able to store `Post` objects.

The UML diagram for the `DynamicArray` class is as follows:



To summarize, each `DynamicArray` object contains the following fields:

`private Post[] data`

- This field will act as the “underlying” data structure of your `DynamicArray`. (a.k.a., the field that is responsible for storing `Post` objects).
- You can assume that this field will *never* be `null`.
- `null` is not a valid element in `DynamicArray`.

`private int size`

- This field represents the number of valid `Post` objects in `data`. A valid `Post` object is simply a `Post` object that exists in `data`.

- You may assume that nothing (other than possibly your own code) would change `size` to be something out of bounds of `data` (i.e., `size >= 0 && size <= data.length` will always evaluate to `true`, unless your code manually sets it to be something else).
- Notably, `size` (number of valid elements) is **not** the same as the capacity (length of `data` : `data.length`) of your `DynamicArray`. The **Example** sections that follow should make this clearer.

The `DynamicArray` class also contains the following instance methods:

`public DynamicArray(int initialCapacity)`

- Initialize the `data` array with a length of `initialCapacity` and set `size` to 0 (as there are 0 valid `Post` objects in `data` at this point).
- If `initialCapacity` is 0, then initialize the `data` array with a "default capacity" of 5 instead.
- You may assume that `initialCapacity` will be greater than or equal to 0.

`public void append(Post post)`

- Add the `Post` object `post` to the end of this `DynamicArray`.
- If `post` is `null`, this method should return without doing anything.
- If this `DynamicArray` is at full capacity (i.e., `size` is equal to the maximum capacity of this `DynamicArray`) then you must do the following *before* adding `post` to the end of this `DynamicArray`:
    - Create a new `Post` array that is *double* the maximum capacity of `data`.
    - Transfer all of `data`'s elements to the newly created array. **Note:** Transferred elements must exist at the same indices that they did in `data`.
    - Assign the `data` field of this `DynamicArray` to this newly created array.

- **Example:** assume we have a `DynamicArray` object called `list` whose `data` field is `[post1, null, null]` and `size` field is `1`. Statements on the left-hand side should evaluate to expressions on the right-hand side.

```
list.data → [post1, null, null] // capacity is 3
list.size → 1
list.append(post2);
list.data → [post1, post2, null] // capacity is 3
list.size → 2
list.append(post3);
list.data → [post1, post2, post3] // capacity is 3
list.size → 3
list.append(post4);
list.data → [post1, post2, post3, post4, null, null] // capacity is 6
list.size → 4
```

`public Post remove(int index)`

- Remove (and return) the `Post` object at the specified index in this `DynamicArray`.
- Remember to update `size` accordingly.
- If `index` is strictly less than 0 or greater than or equal to the `size` of this `DynamicArray`, return `null`.
- Since `null` is not a valid element in a `DynamicArray`, `null` **should not** exist in the first `size` indexes in `data` after a removal. Notice how `post3` was *shifted* to the left after the removal of `post2` in the below example.

- **Example**: picking up where we left off in the previous example

```
list.data → [post1, post2, post3, post4, null, null] // capacity is 6
list.size → 4
list.remove(3) → post4
list.data → [post1, post2, post3, null, null, null] // capacity is 6
list.size → 3
list.remove(1) → post2
list.data → [post1, post3, null, null, null, null] // capacity is 6
list.size → 2
```

`public Post get(int index)`

- Return the `Post` object at the specified index
- If `index` is strictly less than 0 or greater than or equal to the `size` of this `DynamicArray`, return `null`.

- **Example**: picking up where we left off in the previous example

```
list.data → [post1, post3, null, null, null, null] // capacity is 6
list.size → 2
list.get(1) → post3
```

`public int size()`

- Return the number of valid `Post` objects in this `DynamicArray`

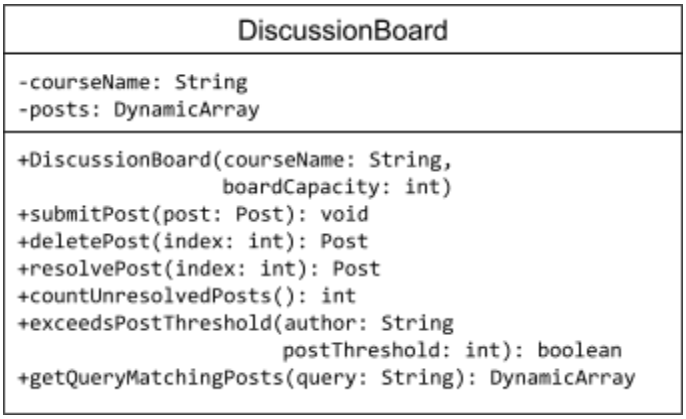
`public int getCapacity()`

- Return the number of `Post` objects that this `DynamicArray` can possibly hold (i.e., the length of `data`)

Part 3: DiscussionBoard.java (30 points)

In this part of the assignment, you will implement a `DiscussionBoard` class which brings together the `Post` class and `DynamicArray` class to create a simple discussion board program.

The UML diagram for the `DiscussionBoard` class is as follows:



To summarize, each `DiscussionBoard` object contains the following fields

`private String courseName`

- The name of the course associated with this discussion board

`private DynamicArray posts`

- A collection of posts associated with this discussion board

The `DiscussionBoard` class also contains the following instance methods:

`public DiscussionBoard(String courseName, int boardCapacity)`

- Initialize the `courseName` field with the value of the `courseName` parameter and initialize the `posts` field to a `DynamicArray`, using `boardCapacity` as the `initialCapacity` argument.
- You may assume that `boardCapacity` will be greater than or equal to 0.

`pubic void submitPost(Post post)`

- Add `post` to the end of this `DiscussionBoard`'s collection of posts.
- If `post` is `null`, this method must return without doing anything.

`public Post deletePost(int index)`

- Remove (and return) the `Post` object at the specified `index` from this `DiscussionBoard`'s collection of posts.

- If `index` is strictly less than 0 or greater than or equal to the `size` of this `DiscussionBoard`'s collection of posts, return `null`.

`public Post resolvePost(int index)`

- Mark (and return) the `Post` at the specified `index` in this `DiscussionBoard`'s collection of posts as resolved. You should be returning the `Post` object *after* marking it as resolved.
- If the `Post` at the specified index is already resolved, return `null`.
- If `index` is strictly less than 0 or greater than or equal to the `size` of this `DiscussionBoard`'s collection of posts, return `null`.

`public int countUnresolvedPosts()`

- Return the number of posts in this `DiscussionBoard` that are unresolved.

`public boolean exceedsPostThreshold(String author, int postThreshold)`

- Return `true` if `author` has created more than `postThreshold` number of posts in this `DiscussionBoard` and `false` otherwise.
- You may assume that `author` will not be `null`.
- You may assume that `postThreshold` is greater than or equal to 0.

`public DynamicArray getQueryMatchingPosts(String query)`

- Return a `DynamicArray` containing posts from this `DiscussionBoard` that contain the string `query` in their `summary` OR `details` fields.
- If `query` is `null`, return `null`
- If there are no posts that contain the string `query` in their `summary` OR `details` fields, return `null`.
- **We strongly encourage you to make use of the String class's `contains()` method.**

Part 4: UnitTest Your Code (10 points)

In this part of the assignment, you will need to implement your own test cases in the method `unitTests` in each of `Post.java`, `DynamicArray.java`, and `DiscussionBoard.java` (i.e., each file has its own `unitTests` method which you must fill out). One simple test case for each of `Post`, `DynamicArray`, and `DiscussionBoard` has already been implemented for you in their respective files. You may use these as inspiration for implementing your own test cases. **To get full credit for this section, you must *exactly* follow the instructions below**

For the `Ticket` class, you must:

- Implement **one** test case for `getDetails()`
- Implement **one** test case for `setDetails(String details)`
- Implement **one** test case for `toString()`

For the `DynamicArray` class, you must

- Implement **one** test case for `get(int index)`
- Implement **one** test case for `append(Post post)`
- Implement **one** test case for `remove(int index)`

For the `DiscussionBoard` class, you must

- Implement **one** test case for `countUnresolvedPosts()`
- Implement **one** test case for `exceedsPostThreshold(String author, int postThreshold)`
- Implement **one** test case for `getQueryMatchingPosts(String query)`

You are of-course welcome (and encouraged) to add more test cases but, as long as you follow the above instructions, you will receive full credit for this part of the assignment.

Submission

You're almost there! Please follow the instructions below carefully and use the **exact submission format**. Because we will use scripts to grade, **<u>you may receive a zero</u>** if you do not follow the same submission format.

1. Open Gradescope and login. Then, select this course → PA5.
2. Click the DRAG & DROP section and directly select the required files `Post.java`, `DynamicArray.java`, and `DiscussionBoard.java`. Please make sure you **<u>DO NOT</u>** submit a zip, just the three files in one Gradescope submission. Make sure the names of the files are correct.
3. You can resubmit unlimited times before the due date. Your score will depend on your final (most recent) submission, even if your former submissions have higher scores.
4. Your submission should look like the below screenshot.

Submit Programming Assignment

Upload all files for your submission

Submission Method

(•) Upload ( ) GitHub ( ) Bitbucket

Add files via Drag & Drop or Browse Files.

Name	Size	Progress	✖
DiscussionBoard.java	**3.4** KB		✖
DynamicArray.java	**2.4** KB		✖
Post.java	**2.9** KB		✖